Latches and Flip-Flops

Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io

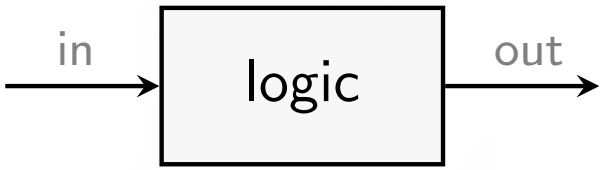


Memory can be thought of as a circuit that remembers its output.

To do this, we feed the output back into the input.

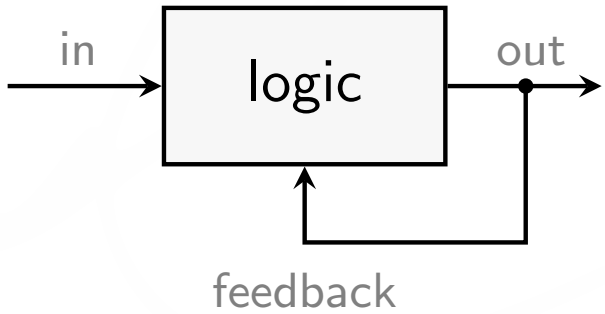
combinational circuit

no output feedback



sequential circuit

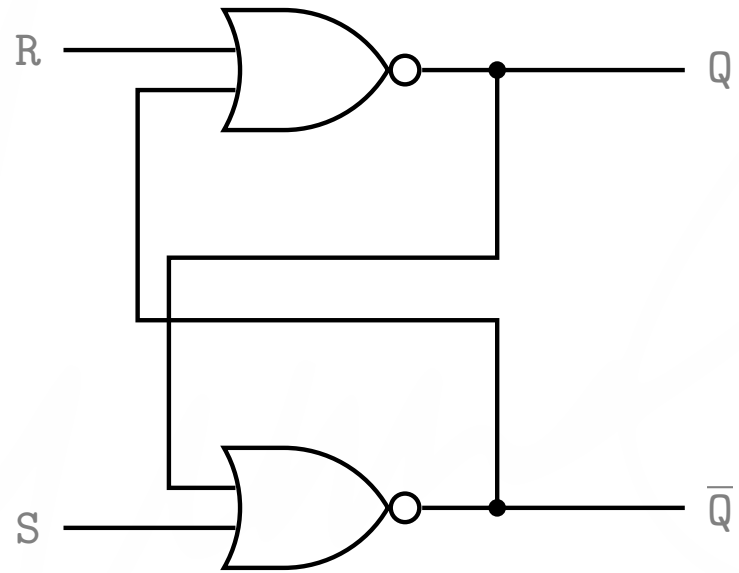
output feedback



The output of a sequential circuit depends on its inputs *and* on what it has stored.

the SR latch

Two cross-coupled NOR gates form an SR **latch**. Each gate's output feeds an input of the other.



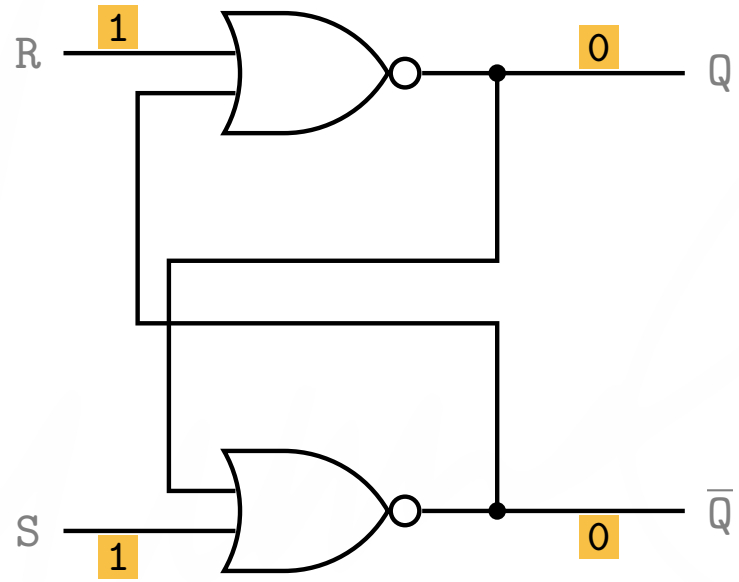
Assuming the two outputs, Q and $\overline{Q}$ are indeed logical inverses, we can derive a timed truth table for this circuit.

The two inputs set (S) and reset (R) the stored bit (Q).

S	R	Q_{t-1}	Q_t	action
0	0	0	0	hold
0	0	1	1	
1	0	0	1	set
1	0	1	1	
0	1	0	0	reset
0	1	1	0	
1	1	0	--	forbidden
1	1	1	--	

The key is that S (resp. R) need not stay 1 for Q to stay 1 (resp. 0).

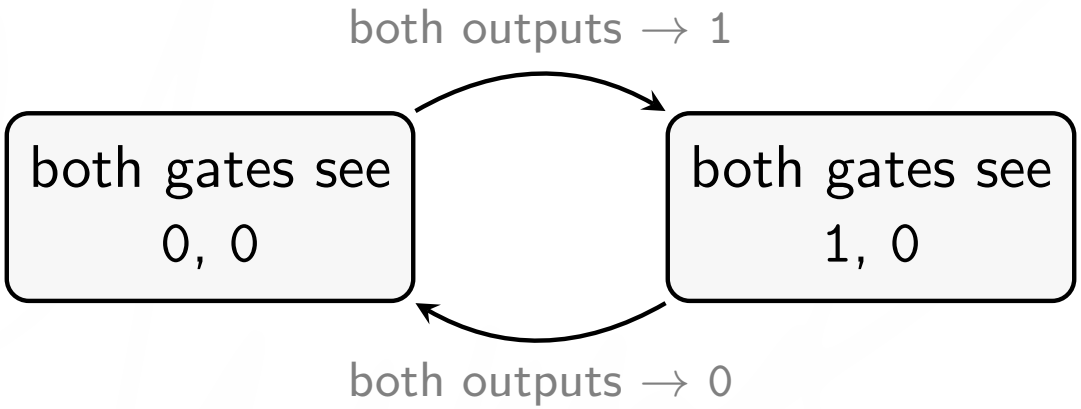
Setting $\mathtt{S} = \mathtt{R} = 1$ is forbidden because it will cause both NOR gates to output 0, which violates the assumption that Q and $\overline{\mathtt{Q}}$ are logical inverses.



Q and $\overline{\mathtt{Q}}$ are both 0: they are no longer opposites.

The latch is now in broken state. Going from this forbidden state directly to the hold state will cause an oscillation.

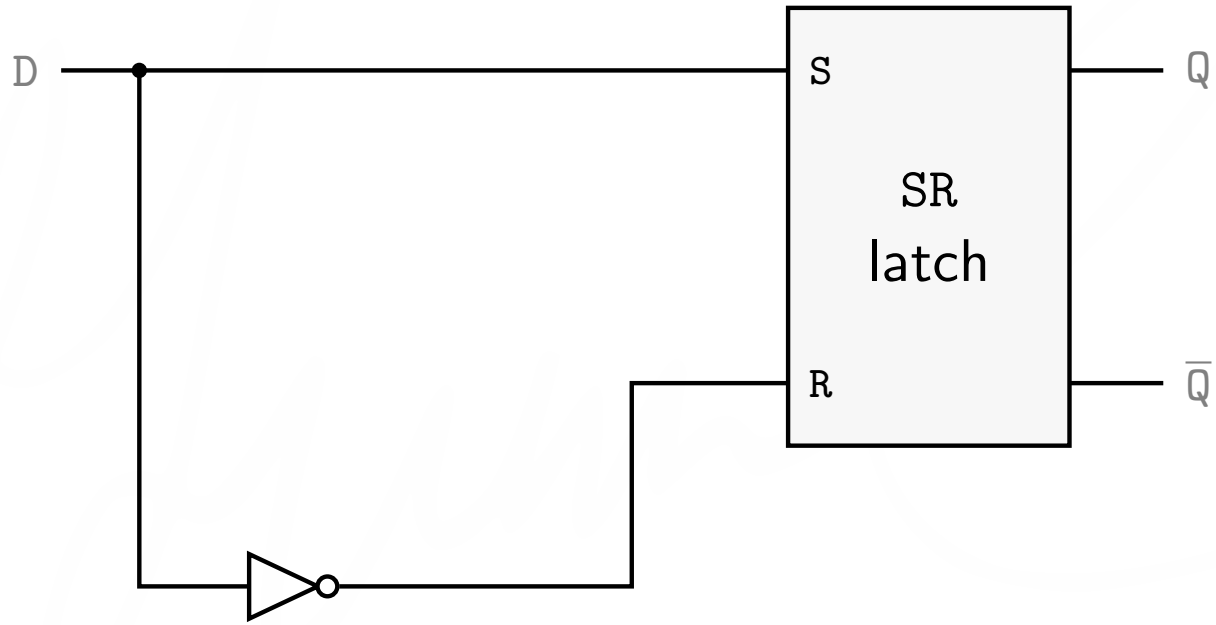
If both inputs reach their gates at exactly the same time, the two gates switch together, over and over:



In a real circuit, tiny differences eventually break the tie, but we cannot say which way the tie will break.

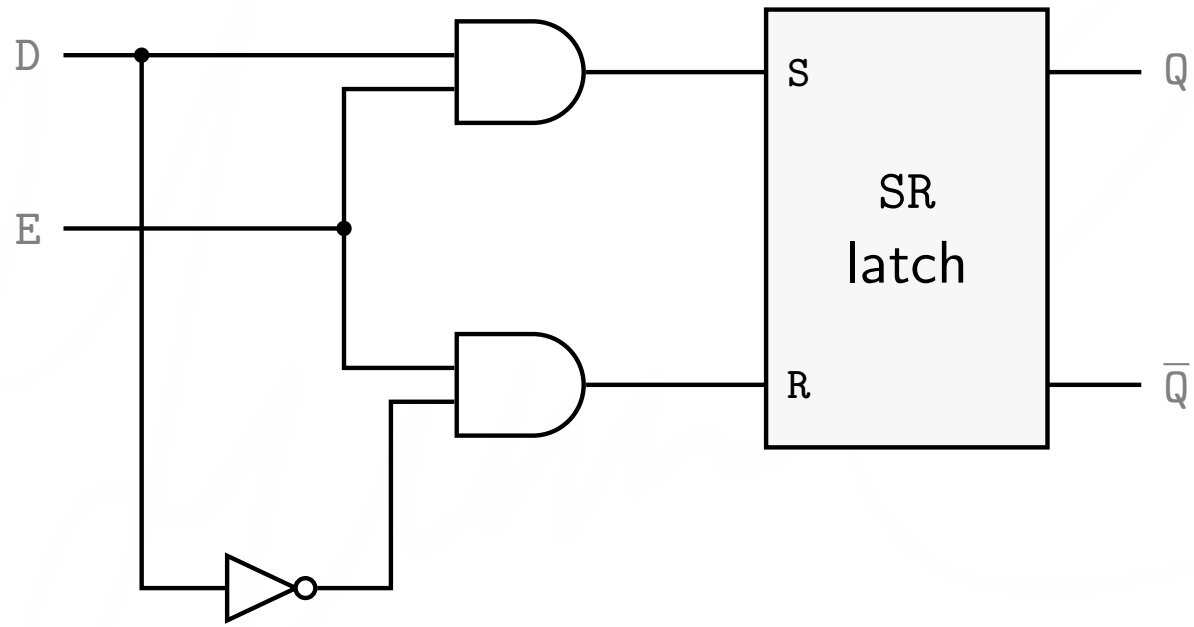
the D latch

To eliminate the forbidden state, we drive both inputs from a single new input, D, with S = D and R = $\overline{\text{D}}$. Now S and R can never both be 1.



But now Q always equals D: the latch can no longer hold.

To let the latch hold, we add an **enable** input, E, and an AND gate on each input. When E = 0, both S and R are forced to 0.



This is a D **latch**.

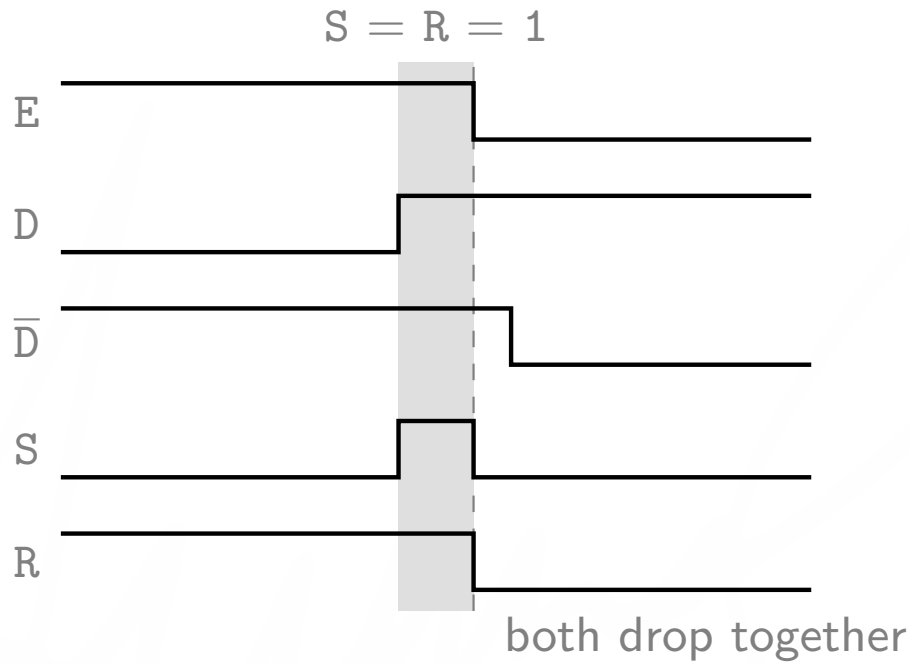
While $\mathtt{E} = 1$, $\mathtt{Q}$ follows $\mathtt{D}$. While $\mathtt{E} = 0$, $\mathtt{Q}$ holds its value.

$\mathtt{D}$	$\mathtt{E}$	$\mathtt{Q}_{t-1}$	$\mathtt{Q}_t$	action
0	0	0	0	hold
0	0	1	1	
1	0	0	0	
1	0	1	1	
0	1	0	0	reset
0	1	1	0	
1	1	0	1	set
1	1	1	1	

No row leads to $\mathtt{S} = \mathtt{R} = 1$, so the forbidden state is gone.

edge-triggering

The D latch only removes the forbidden state once its signals settle. The inverter making $\overline{D}$ takes time, so when D rises, $\overline{D}$ is still 1 for a moment.

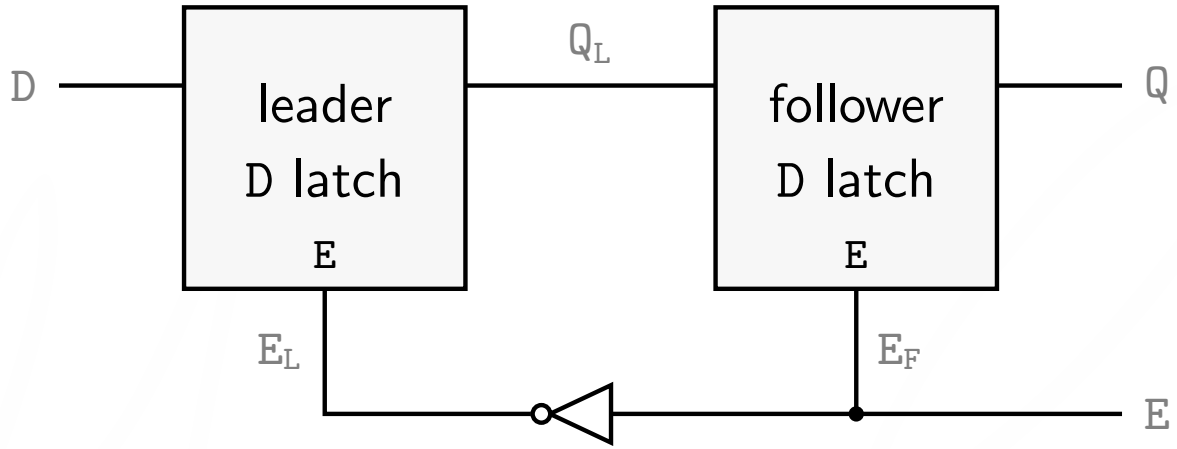


If E drops inside that window, S and R fall from 1 to 0 together, and the latch oscillates exactly as before.

To fix this, we want the circuit to act only at the instant E changes. This is called **edge-triggering**.

the D flip-flop

To build one, we chain two D latches with opposite enables.

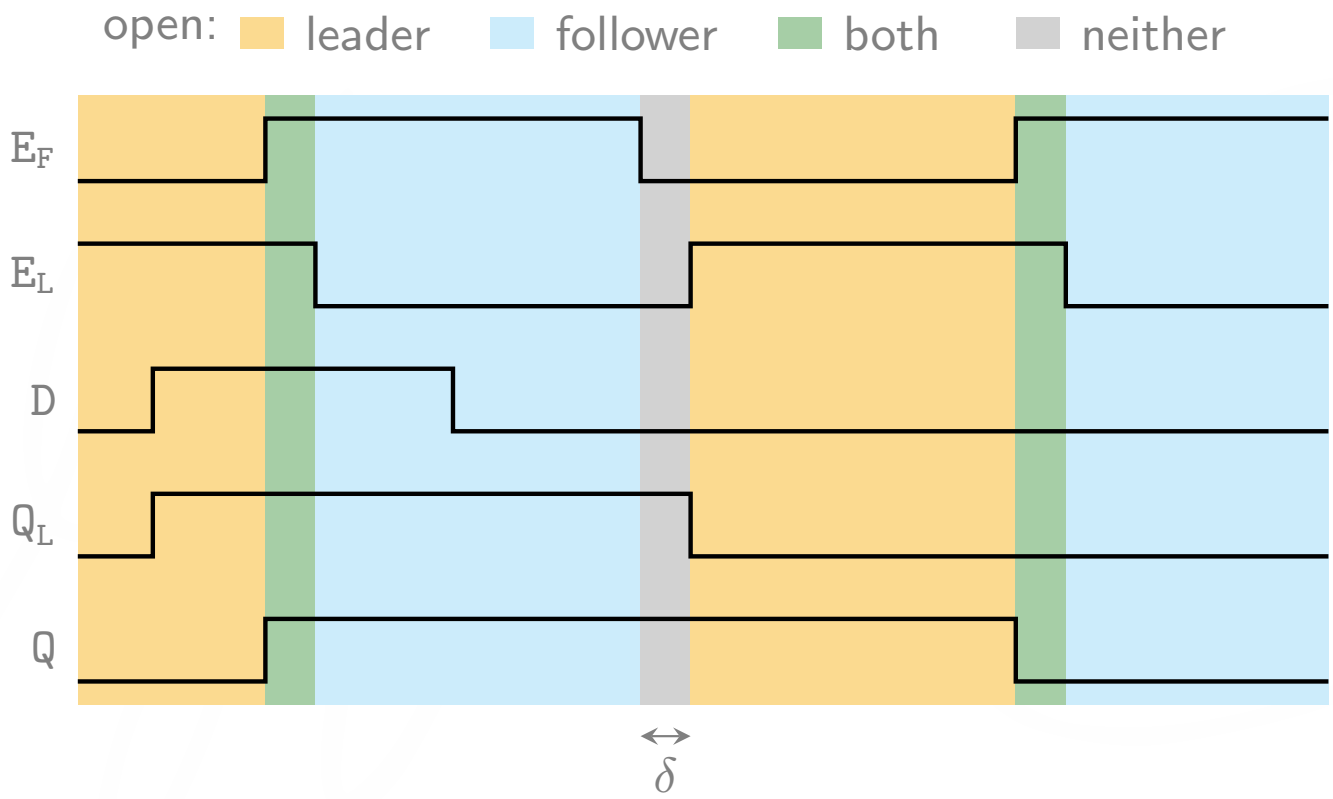


E_F is just E; E_L is E through an inverter

The first, the **leader**, is enabled while $E = 0$. The second, the **follower**, is enabled while $E = 1$.

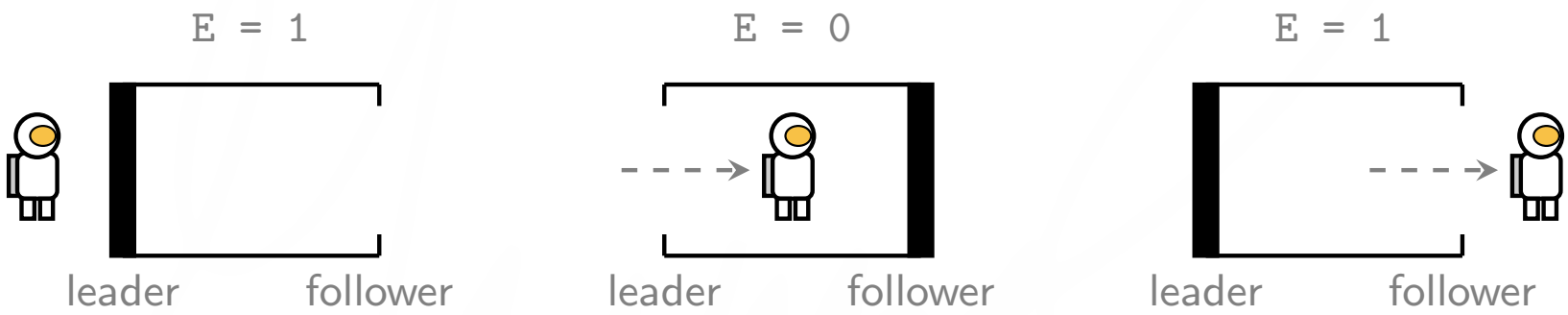
The result is called a **D-flip-flop**.

The inverter takes time, so E_L lags E_F by one gate delay, δ.



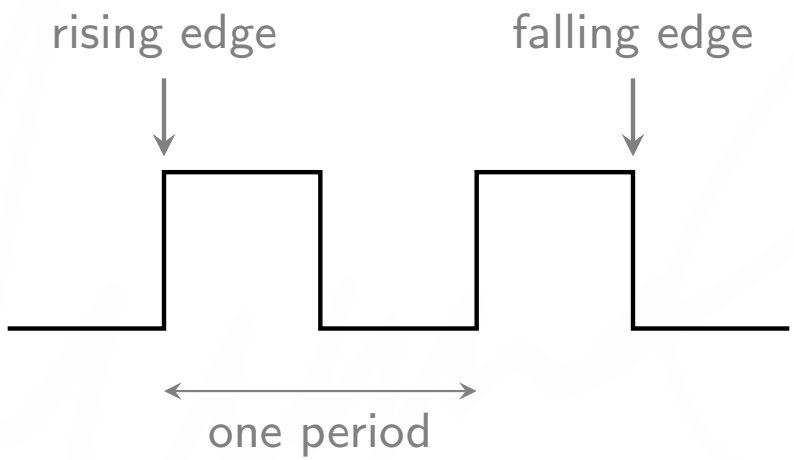
Thus, Q_L changes when E falls, but Q only when E rises. The effect is that any change in D is only reflected after E goes from 0 to 1.

Think of the **leader** and **follower** as two airlock doors. Ideally, they are never open at once, or the cabin depressurizes.



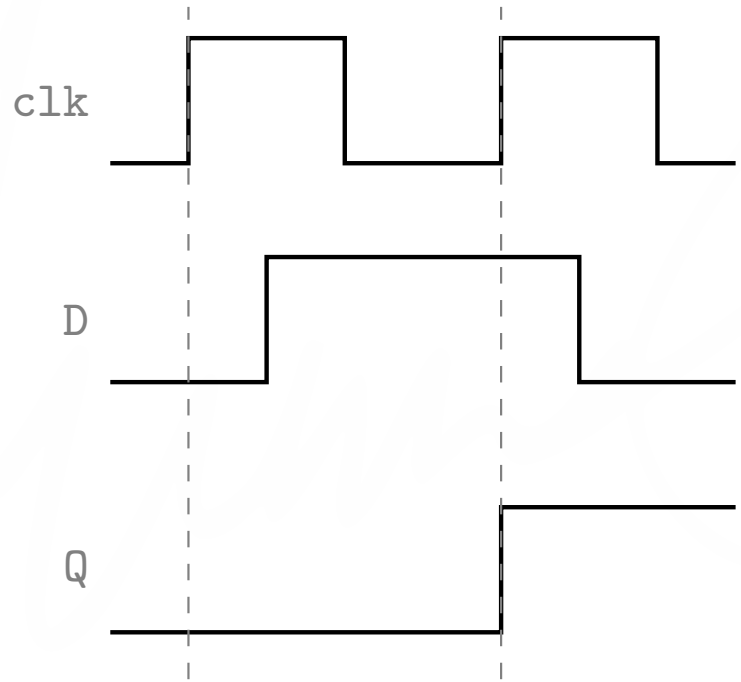
the astronaut steps in while $E = 0$, and out only once E rises

In practice, `E` is usually a **clock** (`clk`): a signal that alternates between `0` and `1` at a steady rate. It tells every part of the circuit when to act.

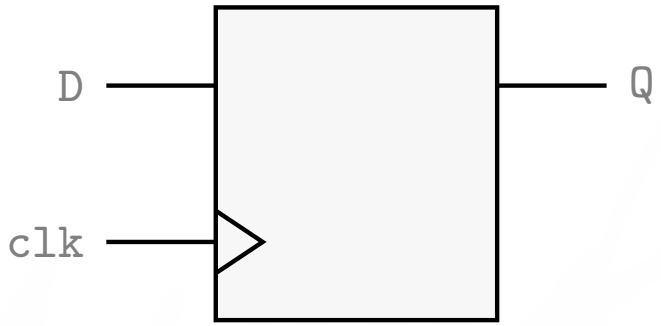


The flip-flop above acts only at each rising edge and ignores `D` in between.

Between edges the output does not change, even if the input does. The flip-flop holds its value until the next rising edge.



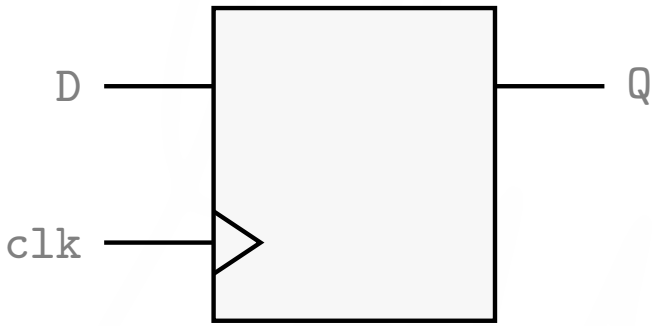
A `D` **flip-flop** stores one bit. On each rising edge of the clock, it copies the input `D` to the output `Q` and holds it.



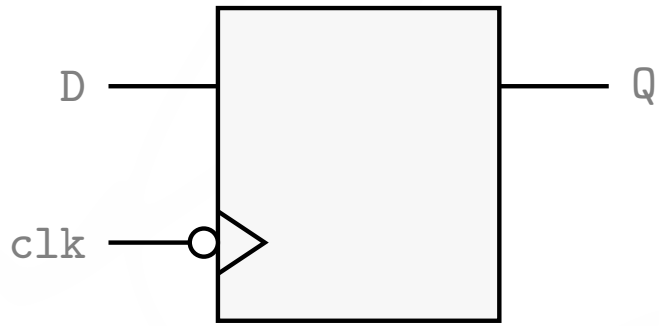
the small triangle marks an edge-triggered clock input

The flip-flop we built acts on the rising (positive) edge, It is also possible to build a flip-flop that acts on the falling (negative) edge.

positive edge

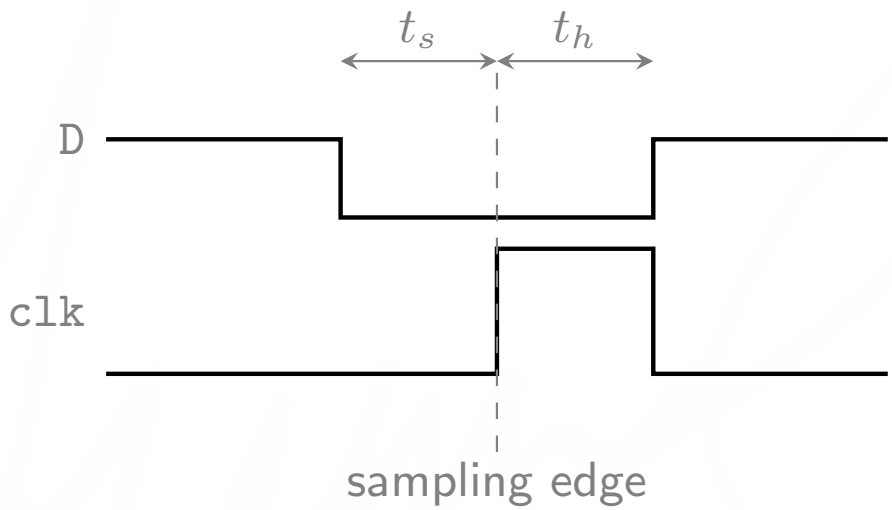


negative edge



the bubble on the clock input means the flip-flop acts on the falling edge

A flip-flop does not capture D instantaneously. Around the clock edge, D must be stable long enough for the flip-flop to reliably capture it.



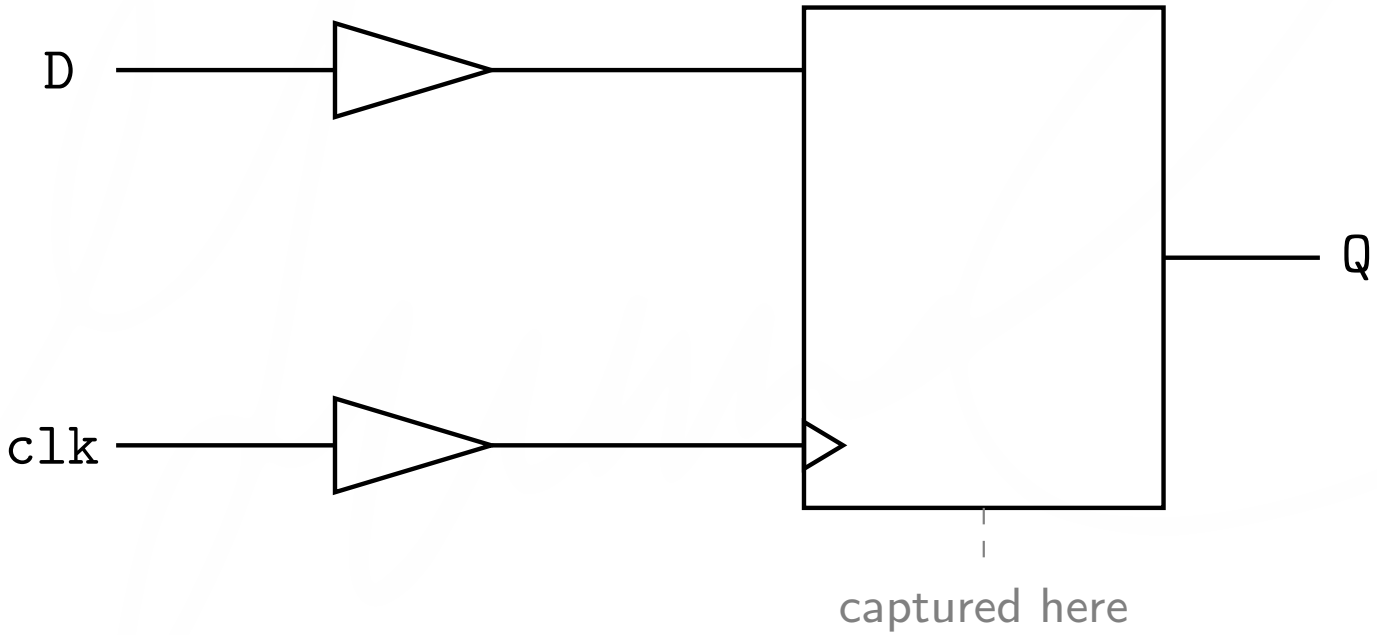
the flip-flop samples D at the clock edge,
stable from t_s before it to t_h after it

setup time

`D` must reach the capture point before `clk` gets there

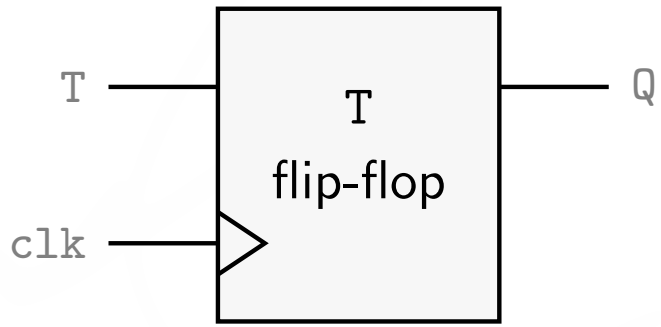
hold time

`clk` must reach the capture point before a new `D` gets there

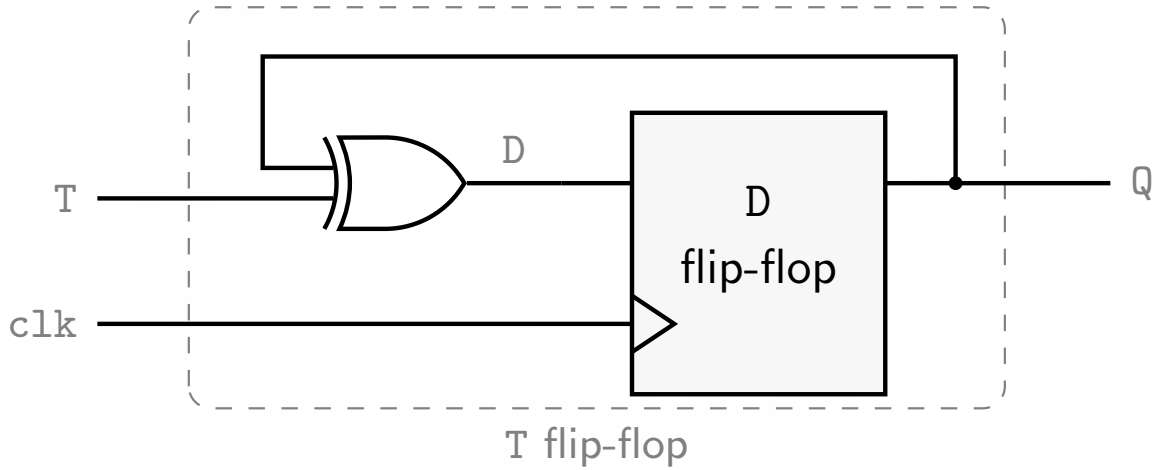


A **T flip-flop** either holds or toggles its stored bit on each rising edge, depending on T.

T	Q_{t-1}	Q_t	action
0	0	0	hold
0	1	1	
1	0	1	toggle
1	1	0	



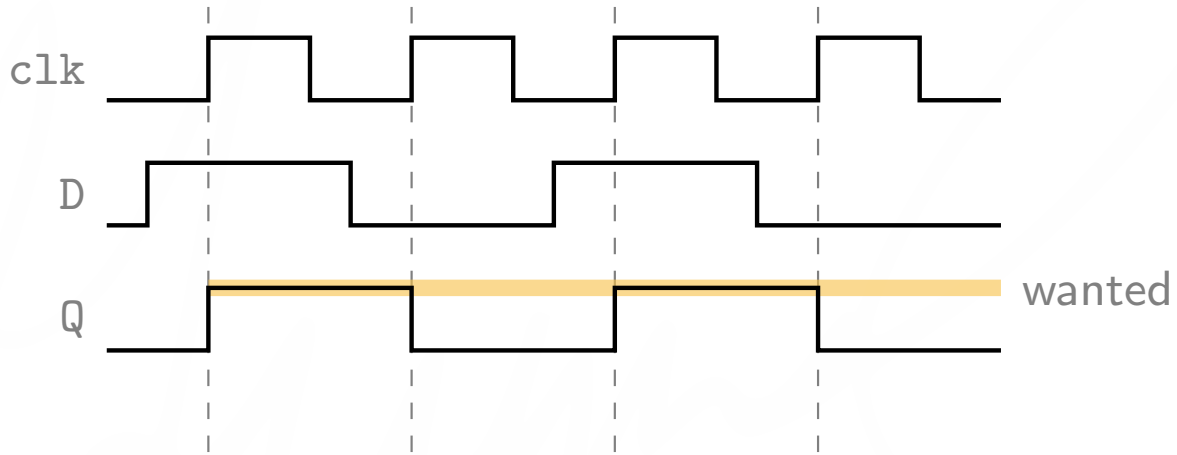
From the table, $Q_t = T \oplus Q_{t-1}$: XOR with 0 keeps a bit, and XOR with 1 flips it. Since a D flip-flop loads D on each edge, we feed it $D = T \oplus Q$.



T flip-flop

when T = 0, the XOR passes Q straight back to D, so the next edge changes nothing

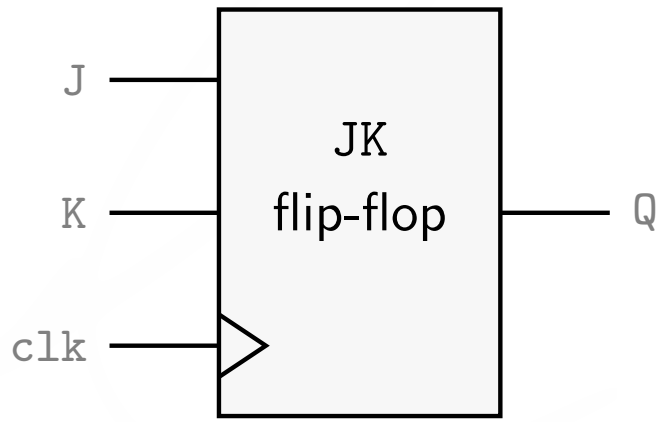
A `D` flip-flop only holds its value until the next edge: **one clock cycle**. On every edge it loads `D` again, whether we want it to or not.



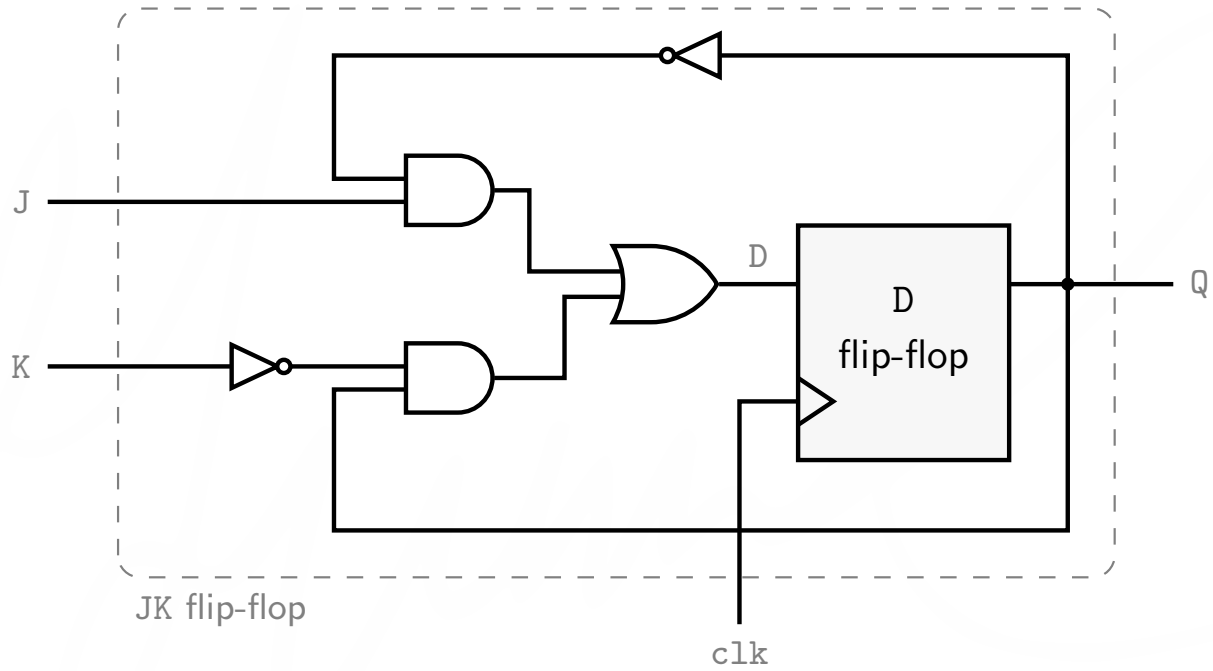
To hold for more than one cycle, `D` must equal `Q` on every edge, so whatever drives `D` has to feed `Q` back in.

A JK **flip-flop** allows us to hold for more than one clock cycle *and* toggle.

J	K	Q_{t-1}	Q_t	action
0	0	0	0	hold
0	0	1	1	
1	0	0	1	set
1	0	1	1	
0	1	0	0	reset
0	1	1	0	
1	1	0	1	toggle
1	1	1	0	



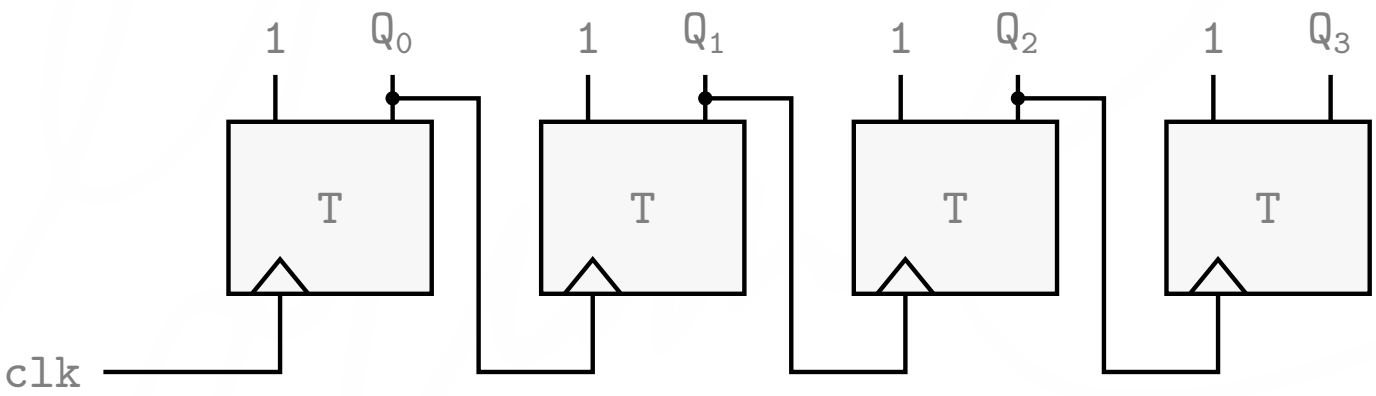
From the table: when $Q_{t-1} = 0$, $Q_t = J$; when $Q_{t-1} = 1$, $Q_t = \overline{K}$. So we feed a D flip-flop $D = J \cdot \overline{Q} + \overline{K} \cdot Q$.



the top AND acts when $Q = 0$ (J decides); the bottom one when $Q = 1$ (K decides)

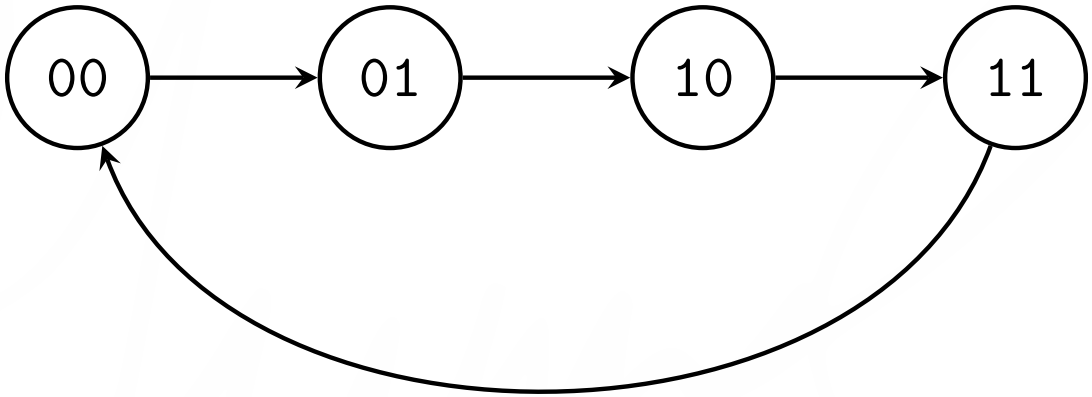
A **counter** is a group of flip-flops whose stored value goes up by one on every clock edge.

The simplest build uses `T` flip-flops, which toggle on each edge. Wiring each output to the next flip-flop's clock makes a binary count.



this ripple counter is cheap, though the carry takes time to reach the top bit

With a fixed number of bits, the count wraps back to zero.



A flip-flop is a one-bit memory. Grouping several of them gives a register, which we build next.